

E | Lab Report

1. Student Information

Lab Number and Title: Lab 1 - Introduction to HDL using Verilog - Logic Gates

Name: Elisha John D. Aton

E-mail: elishajohn.aton@g.msuiit.edu.ph

Lab Section: M89

2. Lab Purpose

This lab introduces the Verilog Hardware Description Language by implementing the OR, NOT, NAND, NOR, XOR, and XNOR gates using three modeling styles: dataflow, behavioral, and structural. Each gate was verified against its truth table using a Verilog testbench and visually verified in GTKWave.

3. Implementation Details

My code structure generally stays the same throughout the lab activity, I used the manual's example code for all of the 3 modeling styles and the testbench, only modifying the lines that needed modifying and the line for the outputs. Rather than separating each style into its own file, I combined all three modules for a given gate into a single Verilog file, and centralized their testbenches into one file that instantiates all three styles at once. This allows me to monitor and verify all three model implementations in one call.

3a. OR Gate (a|b)

Implementation

```
// OR Gate - Dataflow
module or_gate_df (
    input a,
    input b,
    output out
);
    assign out = a | b;
endmodule

// OR Gate - Behavioral
module or_gate_bh (
    input a,
    input b,
    output reg out
);
    always @(*) begin
        out = a | b;
    end
endmodule

// OR Gate - Structural
module or_gate_st (
    input a,
    input b,
    output out
);
    or or1 (out, a, b);
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module or_gate_tb;
    reg a, b;
    wire out_df, out_bh, out_st;

    or_gate_df dut_df (
        .a(a),
        .b(b),
        .out(out_df)
    );

    or_gate_bh dut_bh (
        .a(a),
        .b(b),
        .out(out_bh)
    );

    or_gate_st dut_st (
        .a(a),
        .b(b),
        .out(out_st)
    );

    initial begin
        $dumpfile("or_gate_tb.vcd");
        $dumpvars(0, or_gate_tb);
        $monitor("Time=%0t ns | a=%b b=%b | out_df=%b out_bh=%b out_st=%b", $time, a, b, out_df, out_bh, out_st);

        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;

        $finish;
    end
endmodule
```

3b. NOT Gate (~a)

Implementation

```
// NOT Gate - Dataflow
module not_gate_df (
    input a,
    output out
);
    assign out = ~a;
endmodule
```

```
// NOT Gate - Behavioral
module not_gate_bh (
    input a,
    output reg out
);
    always @(*) begin
        out = ~a;
    end
endmodule

// NOT Gate - Structural
module not_gate_st (
    input a,
    output out
);
    not not1 (out, a);
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module not_gate_tb;
    reg a;
    wire out_df, out_bh, out_st;

    not_gate_df dut_df (
        .a(a),
        .out(out_df)
    );

    not_gate_bh dut_bh (
        .a(a),
        .out(out_bh)
    );

    not_gate_st dut_st (
        .a(a),
        .out(out_st)
    );

    initial begin
        $dumpfile("not_gate_tb.vcd");
        $dumpvars(0, not_gate_tb);
        $monitor("Time=%0t ns | a=%b | out_df=%b out_bh=%b out_st=%b",
$time, a, out_df, out_bh, out_st);

        a = 0; #10;
        a = 1; #10;
```

```

        $finish;
    end
endmodule

```

3c. NAND Gate (~(a&b))

Implementation

```

// NAND Gate - Dataflow
module nand_gate_df (
    input a,
    input b,
    output out
);
    assign out = ~(a & b);
endmodule

// NAND Gate - Behavioral
module nand_gate_bh (
    input a,
    input b,
    output reg out
);
    always @(*) begin
        out = ~(a & b);
    end
endmodule

// NAND Gate - Structural
// Built from an AND gate with its output feeded to a NOT gate
module nand_gate_st (
    input a,
    input b,
    output out
);
    wire and_out;

    and and1 (and_out, a, b);
    not not1 (out, and_out);
endmodule

```

Testbench

```

`timescale 1ns / 1ps

module nand_gate_tb;
    reg a, b;
    wire out_df, out_bh, out_st;

    nand_gate_df dut_df (
        .a(a),
        .b(b),

```

```

        .out(out_df)
    );

    nand_gate_bh dut_bh (
        .a(a),
        .b(b),
        .out(out_bh)
    );

    nand_gate_st dut_st (
        .a(a),
        .b(b),
        .out(out_st)
    );

    initial begin
        $dumpfile("nand_gate_tb.vcd");
        $dumpvars(0, nand_gate_tb);
        $monitor("Time=%0t ns | a=%b b=%b | out_df=%b out_bh=%b out_st=%b", $time, a, b, out_df, out_bh, out_st);

        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;

        $finish;
    end
endmodule

```

3d. NOR Gate ($\sim(a|b)$)

Implementation

```

// NOR Gate - Dataflow style
module nor_gate_df (
    input a,
    input b,
    output out
);
    assign out = ~(a | b);
endmodule

// NOR Gate - Behavioral style
module nor_gate_bh (
    input a,
    input b,
    output reg out
);
    always @(*) begin
        out = ~(a | b);
    end
endmodule

```

```

        end
    endmodule

// NOR Gate - Structural style
// Built from an OR gate with its output feeding a NOT gate.
module nor_gate_st (
    input a,
    input b,
    output out
);
    wire or_out;

    or or1 (or_out, a, b);
    not not1 (out, or_out);
endmodule

```

Testbench

```

`timescale 1ns / 1ps

module nor_gate_tb;
    reg a, b;
    wire out_df, out_bh, out_st;

    nor_gate_df dut_df (
        .a(a),
        .b(b),
        .out(out_df)
    );

    nor_gate_bh dut_bh (
        .a(a),
        .b(b),
        .out(out_bh)
    );

    nor_gate_st dut_st (
        .a(a),
        .b(b),
        .out(out_st)
    );

    initial begin
        $dumpfile("nor_gate_tb.vcd");
        $dumpvars(0, nor_gate_tb);
        $monitor("Time=%0t ns | a=%b b=%b | out_df=%b out_bh=%b out_st=%b",
$time, a, b, out_df, out_bh, out_st);

        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
    end
endmodule

```

```

        a = 1; b = 0; #10;

        a = 1; b = 1; #10;

    $finish;
end
endmodule

```

3e. XNOR Gate ($\sim(a \wedge b)$)

Implementation

```

// XNOR Gate - Dataflow
module xnor_gate_df (
    input a,
    input b,
    output out
);
    assign out = ~(a ^ b);
endmodule

// XNOR Gate - Behavioral
module xnor_gate_bh (
    input a,
    input b,
    output reg out
);
    always @(*) begin
        out = ~(a ^ b);
    end
endmodule

// XNOR Gate - Structural
// Built by feeding the output of an XOR Gate to a NOT Gate
module xnor_gate_st (
    input a,
    input b,
    output out
);
    wire xor_out;

    xor x1 (xor_out, a, b);
    not (out, xor_out);
endmodule

```

Testbench

```

`timescale 1ns / 1ps

module xnor_gate_tb;
    reg a, b;

    wire out_df, out_bh, out_st;

```

```

    xnor_gate_df dut_df (
        .a(a),
        .b(b),
        .out(out_df)
    );

    xnor_gate_bh dut_bh (
        .a(a),
        .b(b),
        .out(out_bh)
    );

    xnor_gate_st dut_st (
        .a(a),
        .b(b),
        .out(out_st)
    );

    initial begin
        $dumpfile("xnor_gate_tb.vcd");
        $dumpvars(0, xnor_gate_tb);
        $monitor("Time=%0t ns | a=%b b=%b | out_df=%b out_bh=%b out_st=%b",
$time, a, b, out_df, out_bh, out_st);

        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;

        $finish;
    end
endmodule

```

3f. XOR Gate ($a \oplus b$) Implementation

```

// XOR Gate - Dataflow
module xor_gate_df (
    input a,
    input b,
    output out
);
    assign out = a ^ b;    // equivalent to (~a & b) | (a & ~b)
endmodule

// XOR Gate - Behavioral
module xor_gate_bh (
    input a,
    input b,
    output reg out

```

```

);

    always @(*) begin
        out = a ^ b;
    end
endmodule

// XOR Gate - Structural
// Built from 2 NOT Gates with their outputs feeding 1 of the inputs of 2
// AND Gates with their outputs feeding both inputs of an OR Gate
module xor_gate_st (
    input a,
    input b,
    output out
);

    wire nota, notb, anda, andb;

    not n1 (nota, a);
    not n2 (notb, b);
    and a1 (andb, a, notb);
    and a2 (anda, b, nota);
    or o1 (out, anda, andb);
endmodule

```

Testbench

```

`timescale 1ns / 1ps

module xor_gate_tb;
    reg a, b;
    wire out_df, out_bh, out_st;

    xor_gate_df dut_df (
        .a(a),
        .b(b),
        .out(out_df)
    );

    xor_gate_bh dut_bh (
        .a(a),
        .b(b),
        .out(out_bh)
    );

    xor_gate_st dut_st (
        .a(a),
        .b(b),
        .out(out_st)
    );

    initial begin

```

```

$dumpfile("xor_gate_tb.vcd");
$dumpvars(0, xor_gate_tb);

$monitor("Time=%0t ns | a=%b b=%b | out_df=%b out_bh=%b out_st=%b",
$time, a, b, out_df, out_bh, out_st);

    a = 0; b = 0; #10;
    a = 0; b = 1; #10;
    a = 1; b = 0; #10;
    a = 1; b = 1; #10;

    $finish;
end
endmodule

```

4. Experimental Results

The Testbench implementations for the OR, NOR, NAND, XNOR, and XOR (Table 4.A) gates follows a uniform input timeline and a separate timeline for the NOT gate (Table 4.B), refer to the tables below:

Table 4.A

	10ns	20ns	30ns	40ns
A	0	0	1	1
B	0	1	0	1

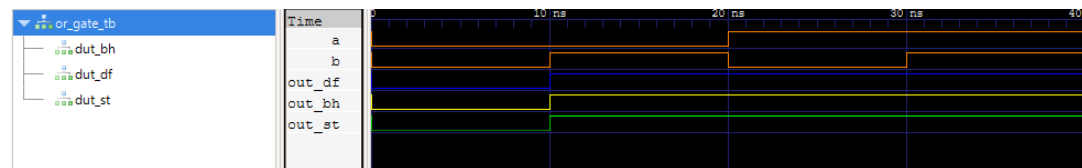
Table 4.B

	10ns	20ns
A	0	1

For easy reference, monitored signals in GTKWave are colored with the following:

- Inputs A & B (a & b) - Orange
- Dataflow Output (out_df) - Blue
- Behavioral Output (out_bh) - Yellow
- Structural Output (out_st) - Green

4a. OR Gate (a|b)

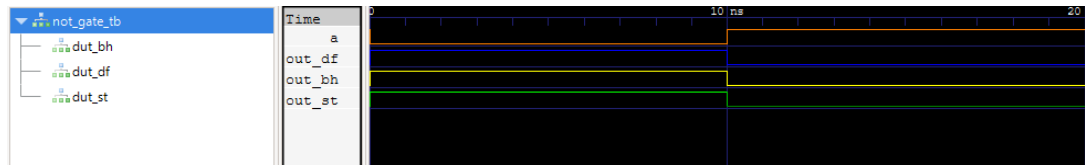


Actual Result:

	10ns	20ns	30ns	40ns
A	0	0	1	1
B	0	1	0	1
OUT (df, bh, st)	0	1	1	1

This matches the expected OR truth table, confirming all three modeling styles behave identically

4b. NOT Gate (~a)

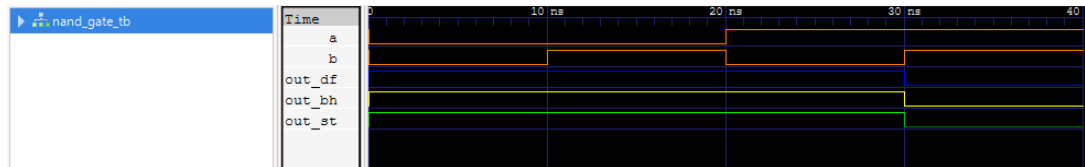


Actual Result:

	10ns	20ns
A	0	1
OUT (df, bh, st)	1	0

This matches the expected NOT truth table, confirming all three modeling styles behave identically

4c. NAND Gate (~(a&b))

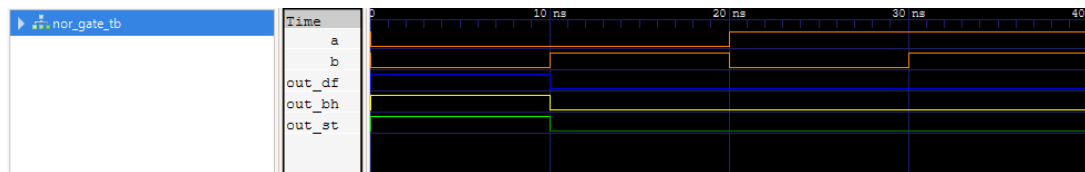


Actual Result:

	10ns	20ns	30ns	40ns
A	0	0	1	1
B	0	1	0	1
OUT (df, bh, st)	1	1	1	0

This matches the expected NAND truth table, confirming all three modeling styles behave identically

4d. NOR Gate (~(a|b))

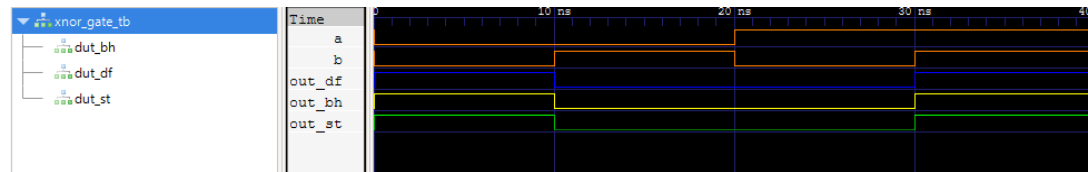


Actual Result:

	10ns	20ns	30ns	40ns
A	0	0	1	1
B	0	1	0	1
OUT (df, bh, st)	1	0	0	0

This matches the expected NOR truth table, confirming all three modeling styles behave identically

4e. XNOR Gate ($\sim(a \wedge b)$)

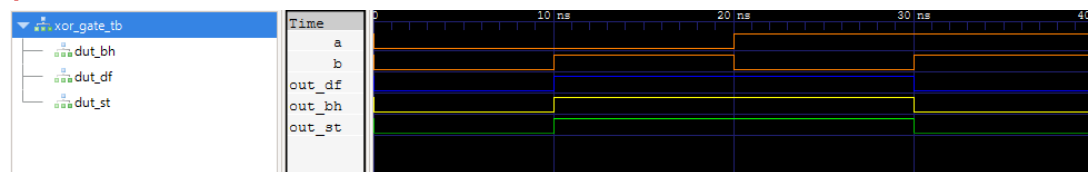


Actual Result:

	10ns	20ns	30ns	40ns
A	0	0	1	1
B	0	1	0	1
OUT (df, bh, st)	1	0	0	1

This matches the expected XNOR truth table, confirming all three modeling styles behave identically

4f. XOR Gate ($a \wedge b$)



Actual Result:

	10ns	20ns	30ns	40ns
A	0	0	1	1
B	0	1	0	1
OUT (df, bh, st)	0	1	1	0

This matches the expected XOR truth table, confirming all three modeling styles behave identically

5. Significance

Understanding logic gates is essential, because they form the foundational knowledge of computing hardware. Using Verilog to model these components rather than relying on manual schematics enables easy development of complex circuits and avoids wasted time by having the ability to simulate the hardware first in a computer before building the actual hardware. Implementing each digital gate in three modeling styles (dataflow, behavioral, and structural) allowed me to understand their practical applications: dataflow handles straightforward assignments efficiently, behavioral modeling simplifies complex conditional logic, and structural designs allow me to reuse modules within larger systems.

6. Pre-lab Questions

1. What is a truth table? - A table that lists every possible combination of inputs and shows what the output should be for each one
2. Which gates are called universal gates and why? - NAND and NOR are considered as universal gate because you can build any other gates using only NAND or NOR gates
3. What is the fundamental difference between an HDL (like Verilog) and a software programming language (like C)? - Software runs top to bottom, one instruction at a time. And Verilog describes hardware that all runs at the same time simultaneously without sequence
4. Define "identifier" in the context of Verilog. - Simply put it is the name given to objects in a Verilog code, say for example a name you can reference/call in your code like an input named "A"
5. A 2-input logic circuit has one HIGH and one LOW input, and its output is HIGH. What type of

- gate is it? - XOR Gate
6. A logic circuit requires all its inputs to be HIGH to make the output HIGH. What type of gate is it? - An AND Gate

7. Post-lab Questions

1. What is meant by "ports" in a Verilog module? - They're the connection points of a module which you can use to connect to other modules or a testbench
2. List the different port modes (e.g., input, output) and their purposes. - Input are for signals that come in, and output is for signals that goes out
3. Describe the different categories of operators in Verilog (e.g., bitwise, logical). - bitwise manipulates corresponding individual bits between signals, and logical evaluates entire signals for true/false conditions
4. What is the difference between blocking (=) and non-blocking (<=) assignment operators? - Blocking assignments executes code sequentially, and non-blocking is the opposite, it executes the code simultaneously
5. What is meant by "simulation" in the context of digital design? - Simply running the design on a computer to verify how it behaves before building it as an actual hardware